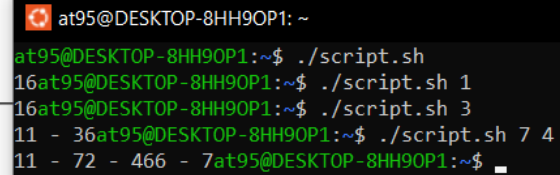


OOS-Find the output of the following code-2022 exams

7. Prikazati izlaz date skripte, u slučaju sljedećih poziva:

- a) ./skripta.sh
- b) ./skripta.sh 1
- c) ./skripta.sh 3
- d) ./skripta.sh 7 4

```
for n in {1..10..5}
do
    echo -n $n
    for k in $*
    do
        if [ $k -gt $n ]
        then
            echo -n "$n - $k"
            n=`expr $n + $n`
        fi
    done
done
```



```
at95@DESKTOP-8HH90P1: ~
at95@DESKTOP-8HH90P1:~$ ./script.sh
16at95@DESKTOP-8HH90P1:~$ ./script.sh 1
16at95@DESKTOP-8HH90P1:~$ ./script.sh 3
11 - 36at95@DESKTOP-8HH90P1:~$ ./script.sh 7 4
11 - 72 - 466 - 7at95@DESKTOP-8HH90P1:~$
```

a)

n je $=1$ jer tako kaže spoljašnja **for** petlja. Zatim se ispisuje vrijednost varijable n te se zbog **-n** unutar **echo** komande ne prelazi u novi red. Varijabla k se inicijalizuje sa 0 jer je **\$*** prazno pošto smo skriptu pozvali bez njoj proslijeđenih argumenata. Ulazimo u **do**-part unutrašnje **for** petlje. Ok, **if** nije ispunjen jer

0 nije greater than 1 tako da preskačemo **echo** i **n=** komande unutar **if** bloka i pošto k nema više vrijednosti za uzimanje istih napuštamo unutrašnju **for** petlju te se vraćamo na gornju. $1+5=6$ tako da je sada $n=6$ i opet ispisujemo to 6 odmah uz prethodno ispisano 1 i ne prebacujemo kursor u novi red. Opet je $k=0$ tako da preskačemo sve unutar unutrašnje **for** petlje jer imamo već opisan scenario zbog $k=0$ te zatim završavamo i unutrašnju pa onda odmah i spoljašnju **for** petlju te nam za ispis ostaje samo 16 .

b) -poklon ako pogledaš i uporediš sa a).

Sve je već opisano u a) jer ako pogledamo u obje iteracije spoljašnje **for** petlje

- 1. **if [\$k -gt \$n] .. da li je 1 greater than 1 ?** nije..
- 2. **if [\$k -gt \$n] .. da li je 1 greater than 1 ?** nije..

tako da je ispis samo 16 .

c)

Spoljašnja **for** petlja inicijalizuje varijablu n sa vrijednošću 1, ispisuje se vrijednost **varijable n** na početni ekran te se ne prelazi u novi red i sada je $k=3$ pošto **\$*** znači redom \$1 \$2 \$3..a mi imamo samo \$1 što je prvi navedeni argument koji je u ovom slučaju jednak broju 3. Uslov u **if**-u je ispunjen jer je $3>1$ tako da se na dosadašnji ispis(1) zbog **echo -n "\$n - \$k"** nadovezuje 1 - 3 pa imamo za sada 11 - 3 te napuštamo unutrašnju **for** petlju jer **\$*** je samo jedan argument tako da automatski se vraćamo na spoljašnju **for** petlju i sada je $n=6$ te se ispisuje 6 kao nadovezica (zbog -n..).

Uslov **if-a** sada nije ispunjen jer 3 nije greater than 6 tako da se skripta završava rezultatom 11 - 36 vidljivim na ekranu.

d)

Spoljašnja **for** petlja inicijalizuje varijablu n sa brojem 1 te ispisuje to na početni ekran. Imamo 1. Unutrašnja **for** petlja te je za prvi argument pozvane skripte $k=7$. Uslov **if-a** jeste ispunjen jer je $7>1$ tako da se na dosadašnji ispis nadovezuje 1 - 7 pa za sada na ekranu imamo 11 - 7 ali se izvršava i dio $n=`expr $n + $n`$ te je $n=2$, a $k=4$. Uslov jeste ispunjen pa se sada ispisuje nadovezica 2 - 4 tako da je sada ispis 11 - 72 - 4 te napuštamo unutrašnju **for** petlju i $n=1+5=6$ jer spoljašnja **for** petlja ponovo inicijalizuje promjenjivu n . Ispisao se vrijednost varijable n tako da sada ispis izgleda ovako 11 - 72 - 46 jer prije ulaska u unutrašnju **for** petlju imamo komandu za ispisivanje nadovezice i onda ulazimo u unutrašnju **for** petlju. OK, $k=7$ i $k>n$ jer je $7>6$ tako da se ispisuje nadovezica 6 - 7 te ispis čini linija 11 - 72 - 466 - 7 i nakon toga $k=4$ što znači da je $k<n$ tako da se zatvara unutrašnja **for** petlja jer zbog neispunjenog uslova preskočene su dvije komande navedene u **if** bloku. n više ne uzima vrijednosti i napušta se skripta tj. završava se i spoljašnja **for** petlja kao i prethodno pokrenuta skripta zajedno sa njom.

7. Prikazati izlaz date skripte, u slučaju sljedećih poziva: (8)

- a) ./skripta.sh
- b) ./skripta.sh 0
- c) ./skripta.sh 1
- d) ./skripta.sh 2

```
a=$#
n=$1
i=1
j=1
while [ $i -le $n ]
do
    j=`expr $j \* $i`
    i=`expr $i + 1`
    echo `expr $j \* $i`
    echo 'expr $j \* $i'
done
echo "$j + $a"
```

```
at95@DESKTOP-8HH90P1: ~
at95@DESKTOP-8HH90P1:~$ ./script.sh
./script.sh: line 5: [: 1: unary operator expected
1 + 0
at95@DESKTOP-8HH90P1:~$ ./script.sh 0
1 + 1
at95@DESKTOP-8HH90P1:~$ ./script.sh 1
2
expr $j \* $i
1 + 1
at95@DESKTOP-8HH90P1:~$ ./script.sh 2
2
expr $j \* $i
6
expr $j \* $i
2 + 1
at95@DESKTOP-8HH90P1:~$
```

a)

a=0 jer je **0** ukupan broj argumenata u ovom pozivu.

n je takođe **=0**, a **j = i = 1** i time smo prve četiri komande skripte već obradili..

Uslov u **while** petlji nije ispunjen tako da se u ovom pozivu samo ispisuje/izvršava komanda "**\$j + \$a**" što je ispisano kao **1 + 0**. Ignorirati poruku vezanu za liniju 5..to je zbog toga što sam skriptu zapisao u Ubuntu builded in Windows 10..

b)

a = 1

n = 0

i = 1

j = 1 i time smo prve četiri komande skripte već obradili..

Uslov u **while** petlji glasi „**Da li je 1 <= 0?**“ -Nije tako da opet izvršava samo poslednja komanda koja na ekran ispisuje

1 + 1

c)

a=1

n=1

i=1

j=1 i time smo prve četiri komande skripte već obradili..

Uslov **while** petlje jeste ispunjen jer je **1 <= 1** tako da ulazimo u tijelo te **while** petlje:

j = j * i = 1 * 1 = 1 ..kome nije jasno zašto neka dobro pročita tekst 9. laboratorijske vježbe!

i = i + 1 = 1 + 1 = 2 ..napominjem da sam ovdje za pojašnjenje koristio jezik svakodnevnice a ne shell script jezik..

e sada u prvoj echo komandi koja slijedi imamo obrnute apostrofe `` te se ispisuje vrijednost proizvoda brojeva 1 koji je pohranjen u promjenjivu j i broja 2 koji je pohranjen u promjenjivu i tj. ispis glasi 2. Zatim u drugoj echo komandi imamo obične apostrofe `` tako da se na naš ekran ispisuje doslovno ono što je napisano između istih expr \$j * \$i i time se završava i naša while petlja tako da za kraj preostaje još jedan ispis a to je zbira vrijednosti pri čime je jedna pohranjena u j ,a druga u a tako da 1 + 1 i ispisaće se 1 + 1 .

d)

a=1

n=2

i=1

j=1 i gle što..time smo prve četiri komande zadate skripte već obradili..

Uslov **while** petlje je ispunjen pa idemo obraditi komande unutar tijela petlje..

1. komanda rezultira sa **j=1**,

2. komanda rezultira sa **i=2**,

3. komanda rezultira ispisom broja **2** na ekran,

4. komanda rezultira ispisom **expr \$j * \$i** na ekran, a zatim je uslov **while** petlje ponovo ispunjen pa idemo redom kroz komande:

1. komanda rezultira sa **j=2**,

2. komanda rezultira sa **i=3**

3.komanda ispisuje broj **6** na ekran,

4. komanda ispisuje **expr \$j * \$i** te se time završava posao while petlje i prelazimo na poslednju komandu u skripti koja još ispisuje **2 + 1** .

7. Prikazati izlaz date skripte, u slučaju sljedećih poziva

- `./skripta.sh`
- `./skripta.sh 1`
- `./skripta.sh 1 2`
- `./skripta.sh 2 1 3`

```
j=$#
for n in {1..3..2}
do
    i=`expr $n + $#`
    echo -n $i
    for k in $*
    do
        if [ $k -lt $n ]
        then
            echo -n "$k"
            n=`expr $n - 2`
        fi
    done
done
```

```
at95@DESKTOP-8HH9OP1: ~
at95@DESKTOP-8HH9OP1:~$ ./script.sh
13at95@DESKTOP-8HH9OP1:~$ ./script.sh 1
241at95@DESKTOP-8HH9OP1:~$ ./script.sh 1 2
351at95@DESKTOP-8HH9OP1:~$ ./script.sh 2 1 3
462at95@DESKTOP-8HH9OP1:~$
```

a) Sve je već poznato iz prvog prethodno urađenog zadatka..

$j=0$ jer smo skriptu pozvali bez eksplicitno predanih argumenata istoj, a znamo da $\$#$ znači ukupan broj argumenata navedenih pri pozivu skripte

, a $n = 1$ što inicijalizuje spoljašnja **for** petlja. Ok, idemo u **do** part spoljašnje **for** petlje..

$i = 1 + 0 = 1$ te će ga ispisati komandom **echo**. $k=0$ jer nismo eksplicitno naveli ništa od argumenata tako da se unutrašnja **for** petlja uopšte i neće izvršavati tako da se i njen blok naredbi preskače i kontrola toka programa se vraća na spoljašnju **for** petlju.

$n=1+2$ && $n \leq 3$ tako da ulazimo u tijelo spoljašnje **for** petlje. Opet $k=0$.. ispisace se vrijednost od i koja je jednaka 3 i naš ispis izgleda ovako **13**.

b)

$j=1$ pošto je poziv skripte popraćen jednim eksplicitno navedenim argumentom..

$n=1$ && $n \leq 3$ tako da ulazimo u tijelo spoljašnje **for** petlje..

$i=1+1=2$ i sljedeće što je jeste to da ga komanda **echo** ispisuje.. Došli smo do unutrašnje **for** petlje koja je k inicijalizovala sa vrijednošću prvog eksplicitno navedenog argumenta pri pozivu skripte tako da je sada $k=1$. Uslov **if**-a unutrašnje **for** petlje nije ispunjen jer isti se pita da li je $1 > 1$ što rezultira sa **false** tako da se kontrola toka programa vraća gore na spoljašnju **for** petlju i sada je $n=1+2=3 \leq 3$ tako da ulazimo u tijelo spoljašnje **for** petlje. Idemo redom komandu po komandu.. $i=4$ i **echo** ga ispisuje pa za sada ispis izgleda ovako **24** i sljedeća komanda k inicijalizuje sa 1 tj. $k=1$. Uslov **if**-a unutrašnje **for** petlje biva ispunjen i izvršavaju se komande u **if** bloku tako da se na ispis nadovezuje broj 1 i ispis za sada izgleda ovako **241**, a $n=n-2=1$ tako da kada se kontrola vrati na vrh unutrašnje **for** petlje uslov u **if** neće biti ispunjen jer su i n i $k = 1$ tako da se završava unutrašnja **for** petlja, a samim tim se završava i spoljašnja **for** petlja.. Ispis: **241**.

c)

$j=2$ ali pošto se j nigdje i ne ispisuje niti **figuriše** u inicijalizaciji bilo koje druge varijable tako da ako se zadatak dobro pogleda j nam je nebitno..

$n=1$ i ulazimo u spoljašnju **for** petlju..Idemo tu redom komandu po komandu..

$i = 1 + 2 = 3$ i zatim se komandom **echo** ta vrijednost varijable i ispisuje na ekran. Ispis: **3**

$k=2$ inicijalizuje ga unutrašnja **for** petlja i samim tim uslov u **if**-u nije zadovoljen tako da se kontrola toka programa vraća na spoljašnju **for** petlju i time je $n=1+2=3 \leq 3$ tako da ponovo ulazimo u tijelo spoljašnje **for** petlje te ćemo komentarisati sve komandu po komandu.

$i = 3 + 2 = 5$ i **echo** ga ispisuje na početni ekran pa izlaz izgleda ovako **35** i unutrašnja **for** petlja inicijalizuje k sa 1 tj. $k=1$ i kako je ispunjen uslov u **if**-u tako se ispisuje i k pa ispis izgleda ovako **351**, a $n=3-2=1$.

Kontrola toka programa prebacuje nas na unutrašnju **for** petlju ali sada je $k=2$ jer je

$\$* \Rightarrow k=\1 pa $k=\$2$ i za takvo k i n uslov **if**-a nije ispunjen te se time zatvara prvo unutrašnja pa zatim i spoljašnja **for** petlja. Konačan ispis: **351**

Napominjem da za poziv skripte sa 1 2 kao argumentima unutrašnja **for** petlja sadrži k i $\$#$ što znači da idemo redom $k=\$1$ pa kada se posao završi $k=\$2$..

d)

Idemo komandu po komandu..

$j=3$

$n=1$

$i=4$ i **ipis: 4**

$k=\$1=2$

if nije ispunjeno pa nas kontrola toka programa vraća na početak unutrašnje **for** petlje..

$k=\$2=1$ te nije ispunjeno **if** kao ni za $k=\$3=3$ tako da nas kontrola toka programa baca na spoljašnju **for** petlju i sada je $n=1+2=3 \leq 3$ pa ulazimo u tijelo spoljašnje **for** petlje..

$i=3 + 3=6$ te ga ispisujemo i ispis glasi: **46**

$k=\$1=2$ i k jeste $< n$ tako da se još ispisuje i 2 te se n inicijalizuje na 1 i nakon toga redom nije ispunjen **if** za $k=\$2$ pa $k=\$3$ tako da konačan ispis glasi: **462**

7. Prikazati izlaz date skripte, u slučaju sljedećih poziva:

- a) ./skripta.sh
- b) ./skripta.sh y
- c) ./skripta.sh a x y z

Maskiranje..

```
x=$#
for x in {10..10}
do
    echo "\"$x\""
    if test $x -le 10
    then
        echo `expr $x + 1`
    fi
done
case $1 in
"x") echo "X";;
"y") echo "Y";;
"z") echo "Z";;
*) echo "DEF";;
esac
```

```
at95@DESKTOP-8HH90P1: ~
at95@DESKTOP-8HH90P1:~$ ./script.sh
"10"
11
DEF
at95@DESKTOP-8HH90P1:~$ ./script.sh y
"10"
11
Y
at95@DESKTOP-8HH90P1:~$ ./script.sh a x y z
"10"
11
DEF
at95@DESKTOP-8HH90P1:~$
```

a)

x=10 što inicijalizuje for petlja. x=\$# se time automatski zamaskira, a ta naredba stoji čisto da zbuni i baci u nedoumicu da li je x= brojEksplicitnoNavedenihArgumenata ili onako kako ga for petlja inicijalizuje.. Idemo redom komandu ko komandu.. Ispisaće se " te zatim vrijednost od x i zatim " što znači da ispis izgleda ovako "10". Uslov u if-u jeste zadovoljen jer je **10 less or equal 10** tako da se odmah ispod dosadašnjeg ispisa ispisuje i **11**. Prilikom poziva skripte nijedan argument joj nije eksplicitno predat tako da se još ispisuje i **DEF**.

b) sve je analogno prethodnom osim što je **case** našao **match** za argument **y** te umjesto DEF imamo izmjenu sa ispisanim **Y**..

c)

Potpuno analogno sa a). Zašto DEF na kraju? Zbog **\$1=a** te X,Y i Z ga ne match-ira i time preostaje samo default slučaj ***)**..

7. Prikazati izlaz date skripte, u slučaju sljedećih poziva

- a) ./skripta.sh
- b) ./skripta.sh 1
- c) ./skripta.sh 2 3
- d) ./skripta.sh 2 1 3

```
for n in {1..4..1}
do
    echo -n $n
    for k in $*
    do
        if [ $k -gt $n ]
        then
            echo -n "$k"
            n=`expr $n / 2`
        fi
    done
done
```

```
at95@DESKTOP-8HH9OP1: ~
at95@DESKTOP-8HH9OP1:~$ ./script.sh
1234at95@DESKTOP-8HH9OP1:~$ ./script.sh 1
1234at95@DESKTOP-8HH9OP1:~$ ./script.sh 2 3
1232334at95@DESKTOP-8HH9OP1:~$ ./script.sh 2 1 3
12132334at95@DESKTOP-8HH9OP1:~$
```

a) ⇔ b)

n=1

Ispis 1

for k in 0

if {} ..skipped

n=n+1=1+1=2 <= 4

Ispis 12

for k in 0

if {} ..skipped

n=n+1=2+1=3 <= 4

Ispis 123

for k in 0

if {} skipped

n=n+1=3+1=4<=4

Ispis 1234

for k in 0

if {} ..skipped

c)

n=1

Ispis 1

for k in 2 3..k=2

if k is greater then n ..true

Ispis 12-k=2 i n = 0

k=3

if k is greater then n ..true

Ispis 123

n=2

Ispis 1232

for k in 2 3

if k is greater then n ..false i onda k=3

if k is greater then n ..true

Ispis 12323 te n=n+1=3 ..

ispis 123233 i if {} ..skipped te n=n+1=4

konačan **ispis 1232334**

d) ..svi koraci su jasni i objašnjeni..

n=1

Ispis 1

for k in 2 1 3

if k is greater then n ..true

ispis 12 i n se inicijalizuje na 0

pa redom za k 1 pa 3 koji se dopisuju ispis izgleda ovako: 1213

te n=1+1=2

Ispis:12132

...samo nastavi i gledaj kako ti izgleda rezultat..

7. Prikazati izlaz date skripte, u slučaju sljedećih poziva:

- a) ./skripta.sh 5
- b) ./skripta.sh 8 5
- c) ./skripta.sh 8 9
- d) ./skripta.sh 9 4

```
if [ $# -ne 2 ]
then
    exit 1
fi
n=`expr $1 - $2`
i=1
until [ ! $i -le $n ]
do
    echo "$n + $i = `expr $i + $n`"
    i=`expr $i + $2`
done
echo $i+n
```

```
at95@DESKTOP-8HH90P1: ~
at95@DESKTOP-8HH90P1:~$ ./script.sh 5
at95@DESKTOP-8HH90P1:~$ ./script.sh 8 5
3 + 1 = 4
6+n
at95@DESKTOP-8HH90P1:~$ ./script.sh 8 9
1+n
at95@DESKTOP-8HH90P1:~$ ./script.sh 9 4
5 + 1 = 6
5 + 5 = 10
9+n
at95@DESKTOP-8HH90P1:~$
```

- a)
1 is not equal with 2 so exit..
- b)
n=8-5=3
i=1
Ispis 3 + 1 = 4
i=1+5=6 što je >n tako da nam još preostaje
Ispis 6+n
- c)
n=8-9=-1
i=1
until je odmah narušen i za ispis nam preostaje još samo poslednja komanda i tako je **ispis: 1+n**
- d)
n=9-4=5
i=1
Ulazimo u until dio..
Ispis: 5 + 1 = 6
i=1+4=5
pa opet ulazimo u until dio..
Ispis: 5+5=10
i=5+4=9 tako da će se until uslov narušiti..
Ispis: 9+n

7. Prikazati izlaz date skripte, u slučaju sljedećih poziva: (8)

- a) ./skripta.sh
- b) ./skripta.sh 1
- c) ./skripta.sh 2
- d) ./skripta.sh 100

```
if [ $# -eq 0 ]
then
    echo exit 1
fi
n=`expr $1 + $#`
i=3
until [ $i -le $n ]
do
    echo "$n * $i = `expr $i \* $n`"
    i=`expr $i - 1`
done
echo $i
```

```
at95@DESKTOP-8HH9OP1: ~
at95@DESKTOP-8HH9OP1:~$ ./script.sh
exit 1
0 * 3 = 0
0 * 2 = 0
0 * 1 = 0
0
at95@DESKTOP-8HH9OP1:~$ ./script.sh 1
2 * 3 = 6
2
at95@DESKTOP-8HH9OP1:~$ ./script.sh 2
3
at95@DESKTOP-8HH9OP1:~$ ./script.sh 100
3
at95@DESKTOP-8HH9OP1:~$
```

a)

Idemo redom komandu po komandu..

*0 jeste equal sa 0 tako da se ispisuje poruka **exit 1***

n=0

i=3

I sve dok ne postane i<=n(=0) vrti se posao u until petlji:

0 * 3 = 0

0 * 2 = 0

0 * 1 = 0

Izašli smo iz until petlje i preostaje nam da još obradimo zadnju komandu koja na naš ekran ispisuje **0**..

b)

Ignoriše se poruka jer 1 nije equal sa 0

n=1+1=2

i=3

i sve dok ne postane i<=n

Ispis 2 * 3 = 6 i dekrementiranjem promjenjive i završava se i posao until petlje tako da još samo poslednja komanda još ispisuje **2**.

c)

n=1+2=3

i=3

Ispis 3

⇔

d)

n=101

i=3

Ispis 3